Lab 03 — Testing SSH Security with Nmap and Hydra

Nenad Damjanović | Cybersecurity — FIŠ | Kali Linux / Docker

Practical laboratory report · 11 November 2026

Objective

Assess an intentionally vulnerable SSH service in an authorized Docker lab. Identify the SSH port, confirm login, test a short password list, and document practical defense measures.

Scope: Local SSH container at 127.0.0.1:2222; the full localhost scan also includes Kali host services. All credentials are fictional lab values.

1. Docker prerequisites

Docker 28.5.2 was available and the Docker service was active. The local TCP port 2222 was free before starting the container.

```
docker --version
sudo systemctl is-active docker
sudo ss -ltnp '( sport = :2222 )'
```



Figure 1. 01-docker-prerequisites.png

2. Instructor Dockerfile inspection

The instructor-provided Ubuntu 22.04 image installs OpenSSH. It deliberately defines testuser / test123 and root / root and enables password authentication and root SSH login. The Dockerfile is vulnerable by design and is suitable only for an isolated, authorized laboratory.

```
cd ~/CS-Vaje-git/lab03/dvws
cat Dockerfile
```

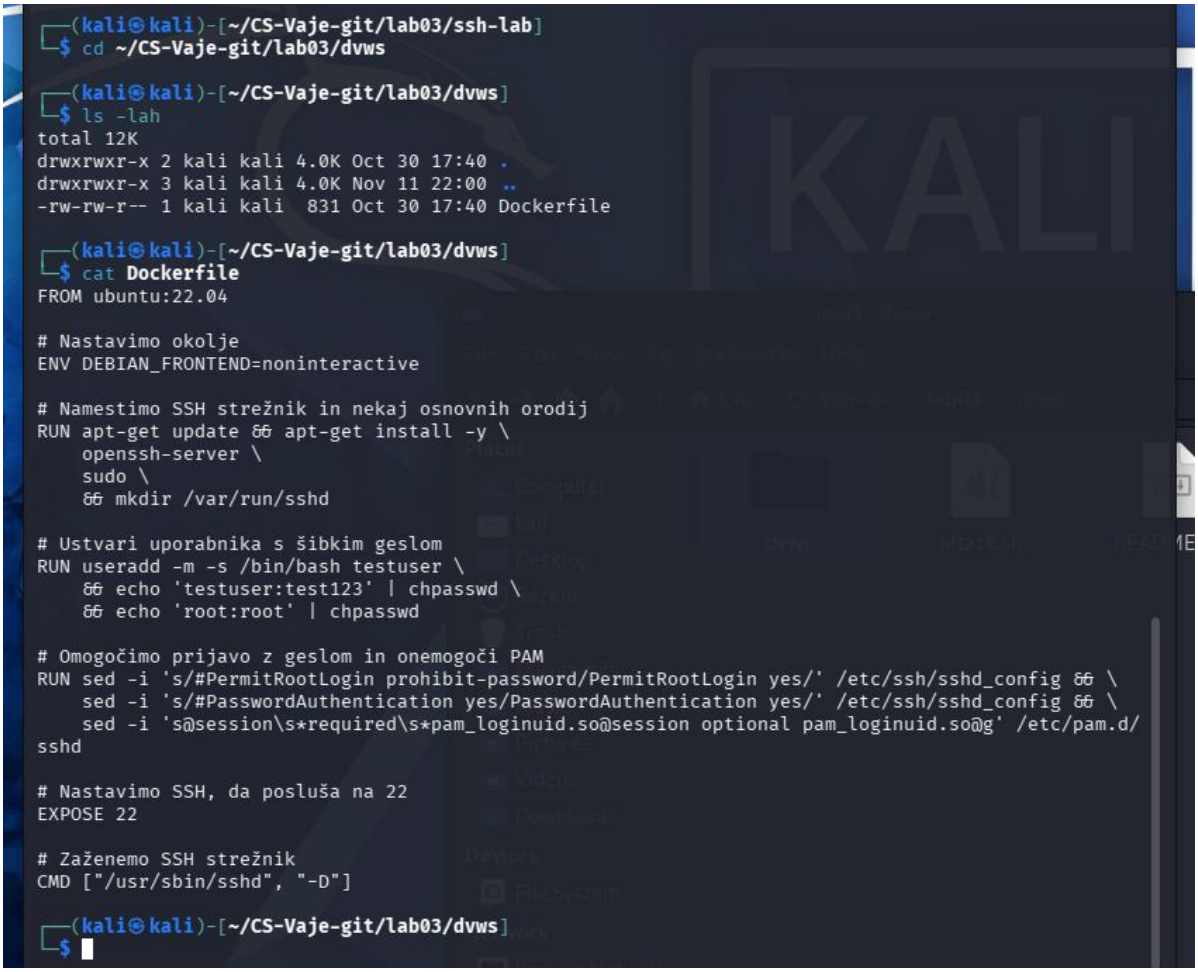


Figure 2. 02-instructor-dockerfile.png

3. Building and starting the SSH container

The image was built as lab03-dvws. An initial attempt to launch the different image name dvws failed; the corrected run command succeeded. The SSH port was published only on host loopback (127.0.0.1:2222), reducing exposure to other machines.

```
sudo docker build -t lab03-dvws .
sudo docker run -d -p 127.0.0.1:2222:22 --name lab03-ssh lab03-dvws
sudo docker ps
sudo ss -ltnp '( sport = :2222 )'
```

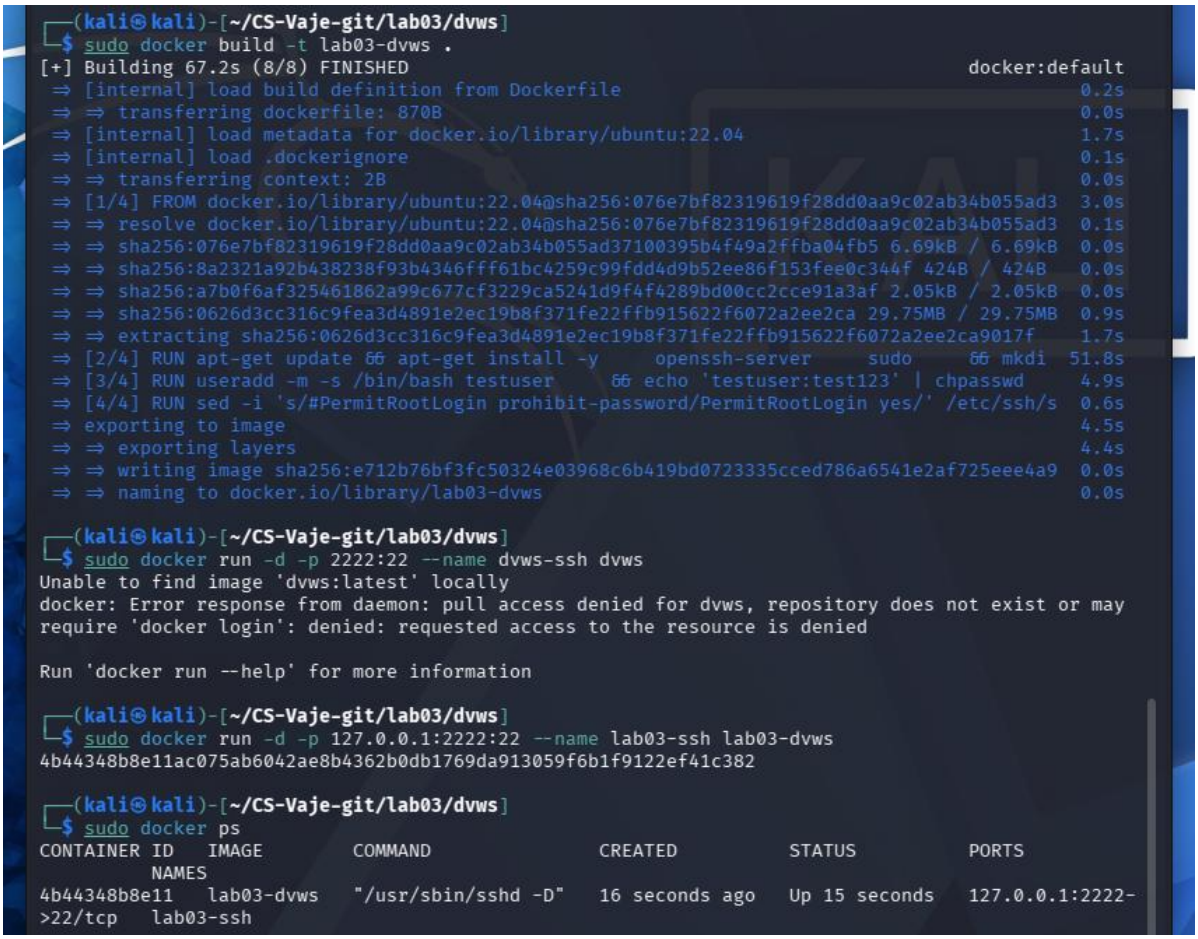


Figure 3. 03-docker-image-and-container.png

4. Nmap service discovery and SSH validation

Nmap found TCP/2222 open and identified OpenSSH 8.9p1 Ubuntu 3ubuntu0.18. A manual SSH connection using the instructor-provided lab account succeeded. The whoami command returned testuser, proving the authentication path worked.

```
nmap -sV -p 2222 127.0.0.1
ssh -p 2222 testuser@127.0.0.1
whoami
hostname
exit
```

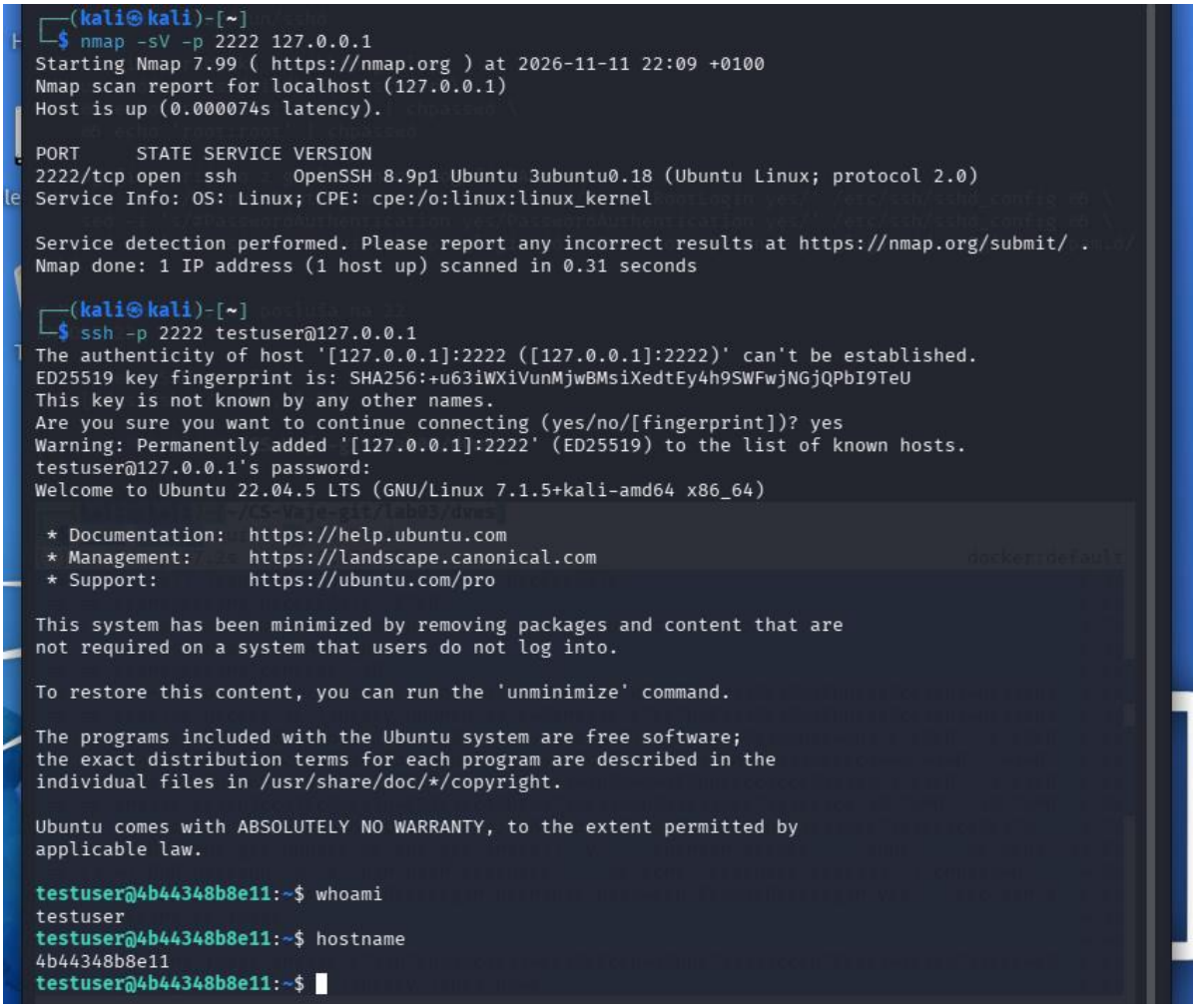


Figure 4. 04-nmap-and-ssh-login.png

5. Controlled password test with Hydra

Hydra 9.7 tested six locally defined password candidates against testuser with one task and stopped after finding test123. The recorded test ran from 22:12:33 to 22:12:47 on 11 November 2026 (14 seconds). This demonstrates that a weak password can be discovered by automated guessing in this controlled setup.

```
hydra -l testuser -P passwords.txt -t 1 -f ssh://127.0.0.1:2222
```

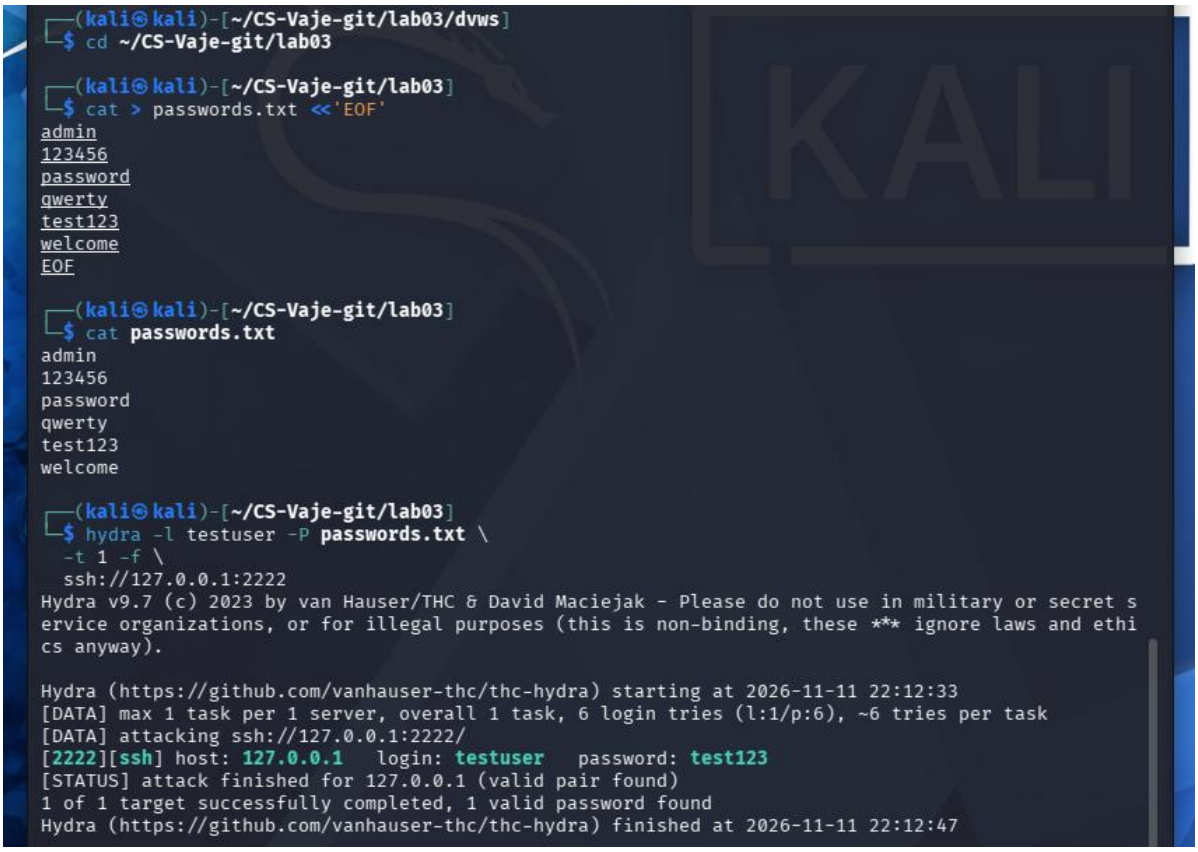


Figure 5. 05-hydra-weak-password.png

6. SSH configuration analysis

The container reported PermitRootLogin yes, PasswordAuthentication yes, and MaxAuthTries 6. The two yes settings are intentionally insecure. For production: verify SSH key access first, then disable root password login and password authentication, restrict source addresses, patch OpenSSH, and monitor authentication attempts.

```
sudo docker exec lab03-ssh grep -E '^(PermitRootLogin|PasswordAuthentication)' /etc/ssh/sshd_config
sudo docker exec lab03-ssh sshd -T | grep -E '^(permitrootlogin|passwordauthentication|maxauthtries)'
```



Figure 6. 06-ssh-security-configuration.png

7. Full TCP scan and limitations

The full localhost TCP scan found port 2222 (SSH) and port 35775 (Golang net/http service). Because the target was 127.0.0.1, the second port belongs to the Kali host environment rather than necessarily to the vulnerable SSH container. Nmap did not produce an exact operating-system match. The SSH banner identifies the container service as Ubuntu OpenSSH, but that is not a reliable fingerprint of the whole host OS.

```
sudo nmap -sS -sV -O -p- 127.0.0.1
```

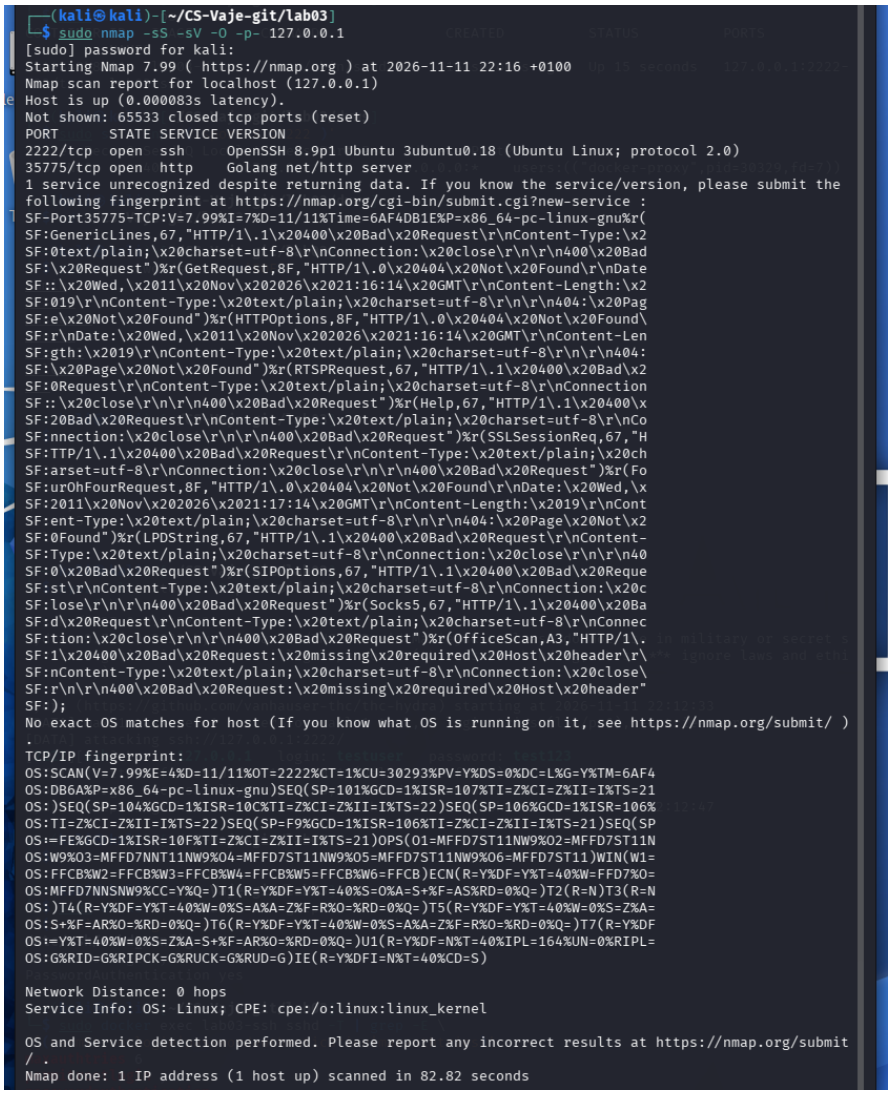


Figure 7. 07-full-nmap-scan.png

Reflection and analysis

How would you protect SSH from brute-force attacks?

Prefer SSH public-key authentication, prohibit direct root login, disable password authentication after testing key access, apply login rate limits and monitor failures.

What additional security measures are recommended?

Apply updates, restrict SSH with host/container firewall rules or a management VPN, use least-privilege user accounts, audit sudo usage, and remove unnecessary listening services.

How does a very strong password change the outcome?

A unique strong password would likely resist this short list of six candidate passwords, but a single unsuccessful test would not prove complete security.

Why close unnecessary open ports?

They enlarge the exposed attack surface. The additional port 35775 was observed on the Kali host during the localhost scan; it cannot be attributed to the SSH container from the presented evidence.

Conclusion

The lab SSH service exposed an intentionally weak password, which Hydra located within a short controlled test. Limiting SSH exposure and replacing password-based access with managed keys are priority defenses.

ChatGPT is used to create a report from my own work and screenshots